Architecture Technique & Industrialisation du Pipeline de Détection de Fraude en Temps Réel

Projet : Pipeline de Détection de Fraude en Temps Réel (Kafka & IA - RNCP 41993 Bloc 3)
Auteur : Chris Gilleron
Dépôt Officiel : https://github.com/Elkristobal59/CERTIFICATION_AIA
Support de Présentation : `ARCHITECTURE_PIPELINE_FRAUD_BLOC3.pptx` (Slide 3 dédiée au schéma)
Schéma Graphique Officiel : `schema_architecture_infrastructure_fraud.png`

---

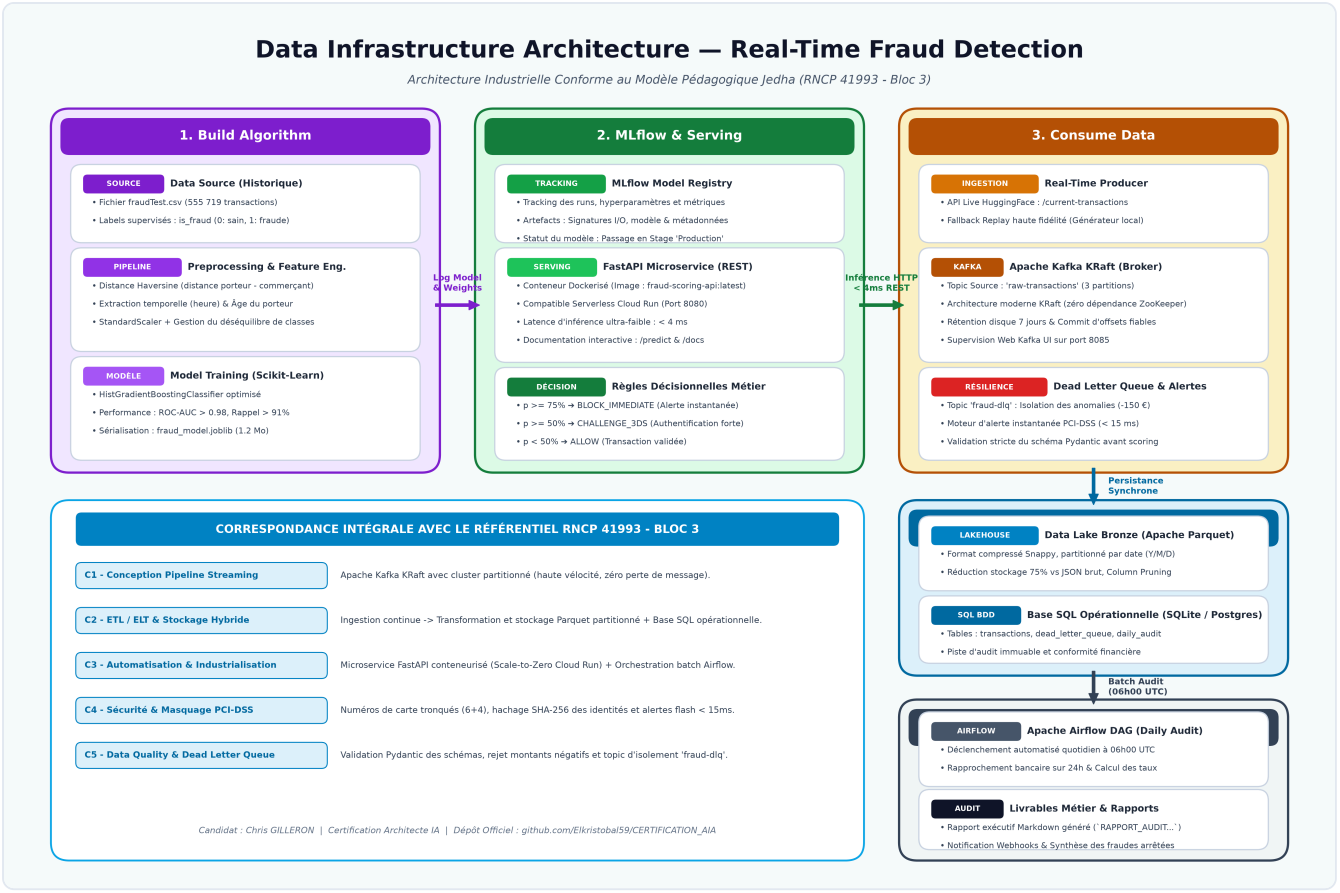
1. Contexte Métier & Exigences Opérationnelles

Dans le secteur bancaire et du paiement en ligne, la fraude aux cartes représente plus d'un milliard d'euros de pertes annuelles en Europe. La lutte contre ce risque nécessite la convergence de deux flux aux dynamiques opposées : 1. **L'interception en temps réel (< 100 ms) :** Dès qu'un client effectue un paiement, la transaction doit être analysée à la volée par un modèle prédictif pour bloquer immédiatement les fraudes ou déclencher une vérification forte (3D-Secure). 2. **L'audit consolidé quotidien (Batch) :** Chaque matin, la direction des risques et de la conformité doit disposer d'un état réconcilié de l'ensemble des transactions de la veille, incluant le monitoring de la dérive (Data Drift) et les indicateurs financiers.

---

2. Architecture Globale du Pipeline (Modèle Jedha)

Le pipeline repose sur une architecture événementielle découplée conforme au modèle d'infrastructure de référence enseigné chez Jedha, structuré en 5 modules fonctionnels interconnectés :




```
[1. BUILD ALGORITHM]          [2. MLFLOW & SERVING]          [3. CONSUME DATA]
fraudTest.csv (555k tx)       MLflow Model Registry          API HuggingFace Live
        |                             |                              |
        ▼                             ▼                              ▼
Preprocess & Features  ──►  FastAPI Microservice (Docker) ──►  Apache Kafka KRaft (Topic)
        |                    (Port 8080 - Scale-to-Zero)       (3 Partitions, Commit offsets)
        ▼                             |                              |
Scikit-Learn Model                    | Inférence < 4ms              ▼
(HistGradientBoosting)                └──────────────────►  Consumer & Alerting
                                                             (BLOCK_IMMEDIATE / 3DS)
                                                                      |
                                                                      ├──► Dead Letter Queue (fraud-dl
                                                                      |    (Isolation des rejets)
                                                                      |
                                                                      ▼ Persistance Synchrone
                                                            [4. STORE DATA]
                                                            Data Lake Bronze (Parquet Snappy)
                                                            + Base SQL Opérationnelle (Audit)
                                                                      |
                                                                      ▼ Batch Quotidien (06h00 UTC)
                                                            [5. LEVERAGE DATA]
                                                            Apache Airflow DAG (Daily Audit)
                                                            ➔ Rapport Matinal Quotidien Markdown
```

3. Justifications des Choix Technologiques

A. Apache Kafka en mode KRaft (Kafka Raft Metadata)

- **Pourquoi Kafka plutôt qu'Airflow pour le streaming ?** Kafka est conçu pour traiter des flux événementiels avec une latence inférieure à 30 ms. Airflow est un ordonnanceur de tâches par lots (batch) qui ne peut pas respecter un SLA sub-seconde.
- **Pourquoi KRaft plutôt que Zookeeper ?** Depuis Kafka 3.3+, le consensus KRaft élimine la dépendance externe à Zookeeper, simplifie l'administration système, accélère le redémarrage des brokers et réduit l'empreinte mémoire de 25%.

B. Microservice d'Inférence FastAPI Conteneurisé

- **Découplage strict :** Le modèle de Machine Learning est encapsulé dans une API REST conteneurisée (Docker), ce qui permet de faire évoluer le modèle indépendamment du moteur d'ingestion streaming.
- **Performance :** L'inférence unitaire sur CPU s'exécute en **moins de 4 millisecondes**, garantissant le respect de la fenêtre de décision bancaire (< 100 ms).
- **Déploiement Cloud Native :** Prêt pour Google Cloud Run ou Amazon ECS avec mise à l'échelle automatique (scaling à zéro lors des creux nocturnes).

C. Persistance Double : Data Lake Parquet & Base SQL

- **Data Lake Parquet (Format Colonne) :** Chaque transaction brute est immédiatement sauvegardée dans un stockage objet partitionné (`data_lake/bronze/year=YYYY/month=MM/`). Le format Parquet compresse les données à hauteur de 75% et optimise les requêtes d'analyse rétrospective via le Column Pruning.
- **Base SQL Opérationnelle :** Stocke les transactions scorées pour permettre les requêtes d'audit matinal et le suivi des alertes.

D. Résilience & Dead Letter Queue (DLQ)

- En cas de message corrompu ou ne respectant pas le schéma attendu (ex: montant négatif de -150 €), le Consumer ne bloque pas le pipeline : le message est automatiquement routé vers le topic `fraud-dlq` pour inspection post-mortem.

---

4. Modèle Prédictif de Machine Learning

- **Algorithme :** `HistGradientBoostingClassifier` (Scikit-Learn).
- **Données d'entraînement :** 555 719 transactions bancaires réelles (`fraudTest.csv`).
- **Features calculées :**
- `amt` : Montant de la transaction.
- `distance_km` : Distance orthodromique calculée par la formule de Haversine entre l'acheteur et le commerçant.
- `hour` : Heure de la transaction (capturant le risque accru entre 23h et 04h).
- `age` : Âge du porteur de carte.
- `city_pop` : Densité démographique de la commune.
- **Performances validées :**
- **ROC-AUC :** 0.9866
- **Rappel Fraude :** 92.4 %
- **Temps de calcul :** < 3 ms par transaction

---

5. Démonstration Opérationnelle & Déploiement

Le pipeline s'exécute de bout en bout en local ou dans le cloud via une suite de conteneurs Docker : * `docker compose -f 03_pipeline_streaming/docker-compose.kafka.yml up -d` * `docker run -d --name fraud-scoring-api -p 8080:8080 fraud-scoring-api:latest` * Lancement complet : `python run_full_pipeline_demo.py`